MNR – Fachliche RAG-Architektur & Implementierungskonzept

Meta-Normen-Register | KI-Plattform für Normendigitalisierung
Version 1.0 | Stand: April 2026
Rolle: Product Owner & Enterprise Architect | Public Sector / Digital Governance

---

1. Überblick & Architekturziele

Das Meta-Normen-Register (MNR) erfordert eine fachagnostische, skalierbare KI-Plattform, die aus normativen Grundlagen (Gesetze, Verordnungen, Standards) automatisiert ein verifizierbares Informationsmodell erzeugt und daraus standardkonforme technische Artefakte ableitet. Die Architektur folgt dem Prinzip **„Law as a System Source"** und bildet eine durchgängige digitale Wertschöpfungskette von der Rechtsnorm bis zum deployten Register.

Architekturgrundsätze

Prinzip	Ausprägung
Open Source Only	Ausschließlich OSS-Komponenten; kein proprietäres Lock-in
Traceability	Jede Aussage im Modell ist auf eine Rechtsquelle zurückführbar
Human-in-the-Middle	Fachaufsicht genehmigt jeden Übergang zwischen Phasen
Fachagnostik	Plattform ist behörden- und registerunabhängig wiederverwendbar
Interoperabilität	XÖV, OpenAPI, JSON-LD, RDF/OWL als Ausgabeformate
Datensouveränität	On-Premise-fähig, vollständig selbst betreibbar

---

2. Zweigeteilter Gesamtprozess

```
┌──────────────────────────────────────────────────────────────┐
│  PHASE 1 – Informationsmodell                                 │
│  Normative Quellen → RAG-Analyse → Informationsmodell (IM)    │
│  → Iterative Fachaufsicht-Freigabe (Human-in-the-Middle)      │
└──────────────────────┬───────────────────────────────────────┘
                       │  Bestätigtes IM (JSON-LD / YAML)
                       ▼
┌──────────────────────────────────────────────────────────────┐
│  PHASE 2 – Artefakt-Generierung                               │
│  Bestätigtes IM → Architektur-Agent → Technische Artefakte    │
```

```
│  (DB-Schema, OpenAPI, Validierungsregeln, Register-Blueprint)   │
└──────────────────────────────────────────────────────────────────┘
```

Beide Phasen sind entkoppelt. Phase 1 kann mehrfach iteriert werden, bis das Informationsmodell durch die Fachaufsicht freigegeben ist. Erst dann wird Phase 2 ausgelöst.

---

3. Architektur-Übersicht (Komponentensicht)

```
┌──────────────────────────────────────────────────────────────────┐
│                        MN-Pflege-Portal (Frontend)                 │
│          Review-UI | Traceability-View | Freigabe-Workflow          |
└──────────────────────────┬──────────────────────────────────────┘
                           | REST/GraphQL
┌──────────────────────────▼──────────────────────────────────────┐
│                        API-Gateway (FastAPI)                       │
│        Routing | Auth (Keycloak/OIDC) | Rate-Limiting | Audit-Log   |
└──────┬──────────────┬──────────────┬──────────────┬──────────────┘
     |              |              |              |
     ▼              ▼              ▼              ▼

┌──────────┐   ┌──────────────┐   ┌────────────┐   ┌────────────────┐
|Ingest-   |   |RAG-Engine    |   |Agenten-    |   |Artefakt-       |
|Pipeline  |   |(LlamaIndex|   |Orches-     |   |Generator       |
|          |   |+ Haystack)|   |trierung    |   |(Phase 2)       |
└────┬─────┘   └──────┬───────┘   └─────┬──────┘   └────────┬───────┘
     |              |              |               |
     ▼              ▼              |               |
┌──────────────────────────────┐    |               |
|   PostgreSQL + pgvector      |◄──┘               |
|   Chunks | Embeddings        |                   |
|   Metadaten | Versionen       |                   |
└──────────────────────────────┘                   |
     |                                             |
     ▼                                             ▼

┌──────────────────┐                    ┌──────────────────────┐
|  Knowledge       |                    |  Artefakt-Store      |
|  Graph           |                    |  (MinIO / S3)        |
|  (Apache Jena    |                    |  JSON, YAML, OpenAPI |
|   / Oxigraph)    |                    |  DB-Schemas          |
└──────────────────┘                    └──────────────────────┘
     |
     ▼

┌──────────────────┐
|  LLM-Backend     |
|  (Ollama/vLLM)   |
```

```
│  Mistral/Llama3 │
└─────────────────┘
```

---

4. Phase 1: Ingest-Pipeline im Detail

4.1 Dokumenten-Ingestion & Parsing

Eingabeformate: PDF, HTML, DOC, DOCX, RTF

Schritt	Aufgabe	Technologie
Dokumenten-Empfang	Upload-API, Dateivalidierung, Versionierung	FastAPI + MinIO
Format-Parsing	Multi-Format-Extraktion (Text, Struktur, Tabellen)	**Apache Tika 2.x** (primär) + **Docling** (IBM OSS, für PDFs mit Struktur)
Struktur-Erkennung	Kapitel, Absätze, §§-Erkennung, Randnummern	Docling + Custom-Parser (Regex + spaCy)
Metadaten-Extraktion	Normtyp, Geltungsbereich, Datum, Jurisdiktion, Version	Regelbasiert + LLM-Klassifikation
Textnormierung	Zeichensatz, Bindestriche, Absätze, Abkürzungen	Custom Python-Pipeline

Metadaten-Schema je Dokument:

```json
{
  "doc_id": "uuid",
  "source_type": "gesetz|verordnung|standard|leitfaden",
  "title": "Meldegesetz BW",
  "jurisdiction": "BW",
  "valid_from": "2024-01-01",
  "valid_to": null,
  "norm_reference": "§ 1 MeldeG BW",
  "version": "2.1",
  "language": "de",
  "register_scope": ["einwohnerregister"],
  "approved_by": "user_id",
  "ingest_ts": "2026-04-16T10:00:00Z"
}
```

4.2 Chunking-Strategie

Das MNR verarbeitet zwei strukturell verschiedene Dokumentklassen, die unterschiedliche Chunking-Strategien erfordern. **Voraussetzung für jede Strategie ist die automatische Dokumenttyp-Erkennung** beim Ingest, die anhand von Strukturmerkmalen (§-Marker, Heading-Hierarchie, Inhaltsverzeichnis) den Chunking-Router steuert.

Dokumenttyp-Erkennung (Chunking-Router)

```
Ingest-Dokument
       |
       ▼
  Struktur-Analyse (Docling + Regex)
       |
  ┌────┴──────────────────────┐
  | §-Marker vorhanden?       | → JA  → Klasse A: Normatives Recht
  | Amtl. Normformat?         |
  └────┬──────────────────────┘
       | NEIN
       ▼
  Heading-Hierarchie vorhanden?
  (H1/H2/H3 oder Nummerierung 1.1.2)?
       |
  ┌────┴──────────────────────┐
  | Klare Kapitelstruktur?    | → JA  → Klasse B: Strukturierte Fach-Dok.
  └────┬──────────────────────┘
       | NEIN
       ▼
  Klasse C: Unstrukturierte Texte (FAQs, Handreichungen)
```

Klasse A: Normative Rechtstexte (Gesetze, Verordnungen, Verwaltungsvorschriften)

Strukturprinzip: §-Paragraphen sind die primäre Sinneinheit. Die Granularität folgt der Rechtshierarchie Paragraph → Absatz → Satz.

Hierarchisches Parent-Child-Chunking:

```
Parent-Chunk (Indexierungsebene für Kontext-Retrieval)
└── § 3 MeldeG BW  [parent_level=1]
    Gesamter Paragraphtext, max. 1.024 Token
    Metadaten: norm_reference="§ 3 MeldeG BW", chunk_type="paragraph"

Child-Chunks (Retrievalebene für präzise Antworten)
├── § 3 Abs. 1 MeldeG BW  [child_level=2]
|   Text des Absatzes, max. 256 Token
|   chunk_type: "tatbestand" | "rechtsfolge" | "definition" | ...
├── § 3 Abs. 2 MeldeG BW  [child_level=2]
└── § 3 Abs. 2 S. 1 MeldeG BW  [child_level=3, optional bei langen Sätzen]
```

Besonderheiten:

- Legaldefinitionen (`„im Sinne dieses Gesetzes"`) erhalten immer einen eigenen Chunk mit `chunk_type="definition"`, auch wenn sie mitten in einem Absatz stehen, da sie für das gesamte Informationsmodell als Begriffe relevant sind
- Verweisstrukturen (`„§ 5 gilt entsprechend"`) werden als eigener Chunk mit `chunk_type="verweis"` und einem expliziten `cross_references`-Feld gespeichert
- Anhänge und Anlagen (z. B. Datenfeldkataloge in Verordnungen) werden tabellarisch als strukturierte JSON-Chunks verarbeitet, nicht als Fließtext

---

Klasse B: Strukturierte Fach-Dokumente (Standards, Fachkonzepte, Leitfäden, Lastenhefte)

Strukturprinzip: Das Inhaltsverzeichnis (ToC) ist die Strukturreferenz. Kapitel und Unterkapitel sind die primären Sinneinheiten – analog zur §-Ebene bei Rechtstexten, aber mit inhaltlicher statt normativer Logik.

Hierarchisches Kapitel-Chunking:

```
Dokument-Chunk (Kontextanker)
└── Kapitel 3: Datenanforderungen  [heading_level=1]
    Titel + Einleitungssatz des Kapitels, 128 Token
    Metadaten: section_path="3", heading="Datenanforderungen"

Abschnitts-Chunks (Retrievalebene)
├── 3.1 Pflichtfelder  [heading_level=2]
|   Vollständiger Text des Unterabschnitts, max. 512 Token
|   chunk_type: "anforderung" | "definition" | "tabelle" | "beispiel"
├── 3.2 Optionale Felder  [heading_level=2]
|   Enthält Tabelle → separater Tabellen-Chunk (JSON-kodiert)
└── 3.2.1 Feldformat Datum  [heading_level=3]
    max. 256 Token
```

Overlap-Behandlung bei Kapitel-Chunks:

- Zwischen Unterkapiteln desselben Kapitels: **15% Token-Overlap** (letzte 2–3 Sätze des vorherigen Chunks werden als Kontext-Präambel angehängt), um inhaltliche Kontinuität (z. B. laufende Definitionen oder Voraussetzungen) zu erhalten
- Zwischen Hauptkapiteln: kein Overlap, stattdessen Kontext über Parent-Chunk-Referenz

Besonderheiten:

- Tabellen werden immer als eigenständige Chunks verarbeitet (JSON-kodiert: Kopfzeile + Zeilen), mit Rückreferenz auf den umgebenden Abschnitts-Chunk; der Abschnitts-Chunk selbst enthält nur die Einleitung/Beschreibung der Tabelle
- Nummerierte Anforderungslisten (typisch in Lastenheften: „A-001: Das System muss …") erhalten pro Anforderung einen eigenen Mini-Chunk mit `requirement_id` im Metadatenfeld, damit einzelne Anforderungen präzise retrievebar sind
- Abbildungen werden mit ihrem Alt-Text/Bildunterschrift als Bild-Chunk gespeichert; der Bildinhalt wird über Docling (OCR/Bildanalyse) in Text überführt, soweit fachlich relevant

---

Klasse C: Unstrukturierte Ergänzungstexte (FAQs, Handreichungen, Auslegungshinweise)

Strukturprinzip: Kein stabiles Strukturgerüst vorhanden. Semantic-Chunking auf Basis inhaltlicher Kohärenz.

- Sentence-Splitter mit semantischer Ähnlichkeitsgrenze (Threshold: cosine similarity < 0,65 → neuer Chunk)
- Min. 3 Sätze, max. 384 Token je Chunk
- 20% Token-Overlap (Sliding Window)
- Diese Dokumente fließen als Auslegungswissen in die Wissensbasis, sind aber in der Traceability als `source_type="auslegung"` markiert und haben eine niedrigere Konfidenz-Gewichtung beim Retrieval als Klasse-A-Quellen

---

Gemeinsames Metadaten-Schema (alle Klassen)

```json
```

```
{
  "chunk_id": "uuid",
  "doc_id": "uuid-ref",
  "doc_class": "A|B|C",
  "source_type": "gesetz|verordnung|standard|fachkonzept|leitfaden|lastenheft|ausleg
  "norm_reference": "§ 3 Abs. 2 MeldeG BW",
  "section_path": "3.2.1",
  "heading_breadcrumb": "Datenanforderungen > Optionale Felder > Feldformat Datum",
  "chunk_type": "tatbestand|rechtsfolge|definition|ausnahme|zustaendigkeit|anforderu
  "hierarchy_level": 2,
  "parent_chunk_id": "uuid-ref",
  "cross_references": ["uuid-ref-2"],
  "text": "...",
  "token_count": 128,
  "overlap_with_prev": 0.15,
  "entities": [],
  "requirement_id": "A-001",
  "confidence_weight": 1.0,
  "version": "2.1",
  "valid_from": "2024-01-01"
}
```

4.3 NLP: Subjekt-Objekt-Prädikat-Extraktion (SVO)

Die Identifikation von **Subjekten, Objekten und Prädikaten** in normativen Texten ist der Kern der semantischen Analyse. Dabei werden folgende Entitäten extrahiert:

```
Normtext: "Die Meldebehörde speichert die Meldedaten der betroffenen Person."
→ Subjekt:   Meldebehörde       [Rolle/Akteur]
→ Prädikat:  speichert          [Pflicht/Handlung]
→ Objekt:    Meldedaten         [Datenobjekt]
→ Kontext:   betroffene Person  [Bezugsobjekt/Rolle]
→ Normtyp:   Pflicht (§ X ...)  [Regeltyp]
```

NLP-Pipeline:

Schritt	Technologie	Modell
Tokenisierung + POS-Tagging	**spaCy 3.x**	`de_core_news_lg`
Named Entity Recognition	**spaCy + Flair**	`de-ner-legal` (custom fine-tuned)
Dependency Parsing (SVO)	**spaCy**	`de_core_news_lg`
Normtyp-Klassifikation	**Hugging Face Transformers**	`deepset/gbert-large` fine-tuned auf Normtypklassen
Semantische Ähnlichkeit	**Sentence-Transformers**	`sentence-transformers/paraphrase-multilingual-mpnet-base-v2`
Koreference Resolution	**spaCy coreferee**	Deutsch-Modell

Klassifikationsschema für Normtypen:

Normtyp	Beispiel	Tag
Pflicht	„muss", „ist zu", „hat … zu"	`MUST`
Ermessen	„kann", „darf", „ist berechtigt"	`MAY`
Verbot	„darf nicht", „ist untersagt"	`MUST_NOT`
Definition	„im Sinne dieses Gesetzes ist"	`DEF`
Ausnahme	„es sei denn", „sofern nicht"	`EXCEPT`
Frist	„binnen", „spätestens", „unverzüglich"	`DEADLINE`
Zuständigkeit	„zuständig ist", „obliegt"	`COMPETENCE`

4.4 Embedding & Vektorspeicherung

Embedding-Modell (primär):

- `multilingual-e5-large` (Microsoft, Apache 2.0)
- Dimension: 1024
- Spezialisiert auf mehrsprachige semantische Suche

Fallback / Spezialisierung:

- `deepset/gbert-large` für rein deutsche Rechtstexte
- Dimension: 1024

pgvector-Schema:

```sql
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE norm_chunks (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  doc_id          UUID NOT NULL REFERENCES norm_documents(id),
  norm_reference  TEXT,
  chunk_type      TEXT,
  hierarchy_level INT,
  parent_id       UUID REFERENCES norm_chunks(id),
  content         TEXT NOT NULL,
  content_hash    TEXT GENERATED ALWAYS AS (md5(content)) STORED,
  entities        JSONB,        -- SVO-Extraktion
  metadata        JSONB,        -- Normtyp, Datum, Geltungsbereich, etc.
  embedding       VECTOR(1024),
  version         TEXT,
  valid_from      DATE,
  valid_to        DATE,
  created_at      TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX ON norm_chunks USING ivfflat (embedding vector_cosine_ops)
  WITH (lists = 100);

CREATE INDEX ON norm_chunks USING gin (metadata);
CREATE INDEX ON norm_chunks (doc_id, norm_reference);
```

---

5. Phase 1: RAG-Engine & Informationsmodell-Generierung

5.1 RAG-Architektur

Framework: LlamaIndex (primär) + Haystack (für Pipelines)

```
Anfrage / Prompt-Suite
        |
        ▼
┌──────────────────────────┐
|  Query-Transformation    |  HyDE, Step-Back, Multi-Query
|  (LlamaIndex)            |
└──────────────────────────┘
           |
           ▼
┌──────────────────────────┐
|  Hybrid Retriever        |  Semantic (pgvector cosine) +
|                          |  Lexikalisch (PostgreSQL FTS)
|  Re-Ranker               |  cross-encoder/ms-marco-MiniLM
└──────────────────────────┘
           |  Top-K Chunks (mit Norm-Referenzen)
           ▼
┌──────────────────────────┐
|  Context Assembly        |  Parent-Child Merge
|                          |  Kontext-Anreicherung (KG-Lookup)
└──────────────────────────┘
           |
           ▼
┌──────────────────────────┐
|  LLM (Ollama/vLLM)       |  Mistral 7B Instruct / Llama 3.1 8B
|  + Structured Output     |  JSON-Mode (Pydantic Schema)
└──────────────────────────┘
           |
           ▼
┌──────────────────────────┐
|  Output Validation       |  Pydantic v2 Schema-Validierung
|  + Traceability Mapper   |  Claim → Zitat → Chunk → Dokument
└──────────────────────────┘
```

5.2 Prompt-Suite (Standard-Abfragen)

Die Prompt-Suite ist die wiederholbare, normierte Abfragemenge, die für jedes Register ausgeführt wird:

Prompt-ID	Ziel	Output-Schema
`PS-01`	Entitäten & Datenobjekte extrahieren	`EntityList`
`PS-02`	Attribute & Datentypen je Entität	`AttributeMap`
`PS-03`	Beziehungen & Kardinalitäten	`RelationGraph`
`PS-04`	Rollen & Zuständigkeiten	`RoleMatrix`
`PS-05`	Pflichten, Verbote, Ermessen	`NormRuleSet`
`PS-06`	Fristen & Prozessregeln	`ProcessConstraints`
`PS-07`	Ausnahmen & Sonderfälle	`ExceptionList`
`PS-08`	Definitionen & Begriffsklärungen	`Glossary`
`PS-09`	Offene Punkte & Interpretationsfragen	`OpenIssues`
`PS-10`	Quellen-Traceability-Matrix	`TraceabilityMap`

Beispiel-Prompt PS-01 (Entitäten):

```
System: Du bist ein Experte für rechtliche Normanalyse im deutschen
Verwaltungsrecht.
Antworte ausschließlich im JSON-Format gemäß dem vorgegebenen Schema.
Keine Erklärungen, keine Markdown-Formatierung.

User: Analysiere die folgenden Normtextauszüge und extrahiere alle fachlichen
Entitäten (Datenobjekte/Registergegenstände), die in einem Register erfasst werden
müssen. Belege jede Entität mit einem konkreten Normzitat.

Kontext:
{retrieved_chunks_with_references}

Schema:
{EntityList_Pydantic_Schema_as_JSON}
```

5.3 Informationsmodell-Struktur (IM)

Das Informationsmodell ist das Kernprodukt von Phase 1. Es ist technologieneutral und prüfbar durch die Fachaufsicht:

```yaml
```

```yaml
# Informationsmodell v1 - Einwohnermelderegister (Beispiel)
im_version: "1.0-draft"
register_name: "Einwohnermelderegister BW"
status: "draft"  # draft | review | approved | rejected
created_by: "rag-agent-v1"
reviewed_by: null
approved_by: null
sources:
  - doc_id: "uuid-1"
    title: "MeldeG BW"
    version: "2024-01-01"

entities:
  - id: "ENT-001"
    name: "Person"
    description: "Natürliche Person, die sich anmeldet"
    source_ref: "§ 1 Abs. 1 MeldeG BW"
    norm_type: "MUST"
    attributes:
      - id: "ATTR-001"
        name: "familienname"
        data_type: "string"
        required: true
        source_ref: "§ 3 Abs. 1 Nr. 1 MeldeG BW"
        constraints: ["max_length: 200", "not_null"]
      - id: "ATTR-002"
        name: "geburtsdatum"
        data_type: "date"
        required: true
        source_ref: "§ 3 Abs. 1 Nr. 4 MeldeG BW"

relations:
  - id: "REL-001"
    from_entity: "Person"
    to_entity: "Adresse"
    type: "HAT_HAUPTWOHNUNG"
    cardinality: "1:1"
    source_ref: "§ 21 MeldeG BW"

rules:
  - id: "RULE-001"
    type: "DEADLINE"
    description: "Anmeldung binnen zwei Wochen nach Einzug"
    expression: "anmeldedatum <= einzugsdatum + 14 Tage"
    source_ref: "§ 17 Abs. 1 MeldeG BW"
```

```yaml
open_issues:
  - id: "OI-001"
    description: "Definition 'unverzüglich' bei § 23 unklar"
    status: "open"
    assigned_to: null

traceability:
  - claim_id: "ATTR-001"
    citation: "Als Grunddaten sind zu speichern: ... Familienname..."
    doc_id: "uuid-1"
    chunk_id: "chunk-uuid-42"
    norm_reference: "§ 3 Abs. 1 Nr. 1 MeldeG BW"
```

---

6. Knowledge Graph

Der Knowledge Graph ergänzt die Vektordatenbank um explizite semantische Relationen:

```
┌──────────────────────────────────────────────────────────┐
│  Apache Jena (RDF-Store) / Oxigraph (leichtgewichtig)    │
│                                                          │
│  Meldebehörde ──[ist_zuständig_für]──▶ Anmeldung        │
│  Person ────────[hat_pflicht]──────▶ Anmeldung           │
│  Anmeldung ─────[hat_frist]────────▶ 14_Tage            │
│  Familienname ─[definiert_in]──────▶ § 3 MeldeG BW      │
│                                                          │
│  Ontologie-Basis: FOAF + Verwaltungsontologie (LIDO/XÖV) │
└──────────────────────────────────────────────────────────┘
```

Einsatz:

- Impact-Analyse bei Gesetzesänderungen
- Explizite Abhängigkeiten zwischen Registern
- Kontextualisiertes Retrieval (Graph-RAG)

---

7. Human-in-the-Middle (Fachaufsicht-Workflow)

```
Agent erzeugt IM-Draft
         │
         ▼
   [MN-Pflege-Portal]
   Fachaufsicht prüft:
   • Vollständigkeit
```

```
    • Korrektheit (mit Quellverweis)
    • Offene Punkte
          |
      ┌───┴───┐
      ▼       ▼
  Freigabe  Ablehnung/Korrektur
      |           |
      |       Ergänzungs-
      |       dokumente
      |       → Wissensbasis
      |       → neue Iteration
      ▼
 Phase 2
```

Workflow-Engine: Apache Airflow DAG (Freigabe-Trigger) + FastAPI Webhook
Audit-Trail: PostgreSQL Tabelle `im_review_log` mit Zeitstempel, User-ID, Aktion, Kommentar
Versionierung: Jeder IM-Stand wird als versioniertes YAML in MinIO archiviert

---

8. Phase 2: Artefakt-Generierung

8.1 Architektur-Agent

Nach Freigabe des IM übernimmt der **Architektur-Agent** die technische Ableitung:

Artefakt	Format	Beschreibung
Datenbankschema	SQL (PostgreSQL)	DDL mit Constraints, Indizes, Fremdschlüsseln
Register-Blueprint	YAML (Factory-DSL)	Metadaten-Schema für Registerfactory
OpenAPI-Spezifikation	JSON/YAML (OpenAPI 3.1)	REST-Schnittstellen für Register-CRUD
Validierungsregeln	JSON / DMN	Fachliche Constraints aus Normen
XÖV-Datenstruktur	XML Schema (XSD)	Für XÖV-konforme Datenübertragung
Rollenmodell	JSON (RBAC)	Zuständigkeiten aus der Normenanalyse
Event-Definitionen	AsyncAPI / CloudEvents	Für ereignisgetriebene Registerarchitektur

8.2 Traceability in Artefakten

Jedes generierte Artefakt trägt Quellenverweise:

```
json
```

```json
{
  "table": "person",
  "column": "familienname",
  "type": "VARCHAR(200)",
  "nullable": false,
  "_sourceRef": {
    "norm": "§ 3 Abs. 1 Nr. 1 MeldeG BW",
    "doc_id": "uuid-1",
    "chunk_id": "chunk-uuid-42",
    "im_claim": "ATTR-001"
  }
}
```

---

9. Vollständiger Technologie-Stack

9.1 Core Infrastructure

Schicht	Komponente	Version / Lizenz
Datenbank	PostgreSQL + pgvector	16+ / Apache 2.0
Objektspeicher	MinIO	AGPL 3.0
Message Queue	Apache Kafka	Apache 2.0
Orchestrierung	Apache Airflow	Apache 2.0
Identität / AuthN	Keycloak (OIDC/OAuth2)	Apache 2.0
Containerisierung	Docker + Kubernetes (K3s)	Apache 2.0
API-Gateway	FastAPI + Nginx	MIT / BSD

9.2 KI / ML Stack

Komponente	Technologie	Modell / Variante
LLM-Serving	Ollama (Dev) / vLLM (Prod)	Mistral 7B Instruct v0.3
LLM (Reasoning)	vLLM	Llama 3.1 8B Instruct
Embedding	Sentence-Transformers via FastEmbed	`multilingual-e5-large`
NLP Pipeline	spaCy 3.x	`de_core_news_lg`

Komponente	Technologie	Modell / Variante
NER (Recht)	Flair + spaCy	Custom fine-tuned auf Verwaltungsrecht
Klassifikation	Hugging Face Transformers	`deepset/gbert-large`
RAG Framework	LlamaIndex 0.10+	-
Pipeline Framework	Haystack 2.x	-
Re-Ranker	Cross-Encoder	`cross-encoder/ms-marco-MiniLM-L-12-v2`
KG-Speicher	Apache Jena Fuseki	Apache 2.0
LLM Observability	Langfuse	MIT

9.3 Frontend / Portal

Komponente	Technologie
MN-Pflege-Portal	Next.js 14 + React
UI-Komponenten	shadcn/ui + Tailwind CSS
API-Client	TanStack Query
Diagramme/IM-Visualisierung	React Flow / Mermaid.js

9.4 Monitoring & Observability

Komponente	Technologie
Metriken	Prometheus + Grafana
Logging	Loki + Grafana
Tracing	OpenTelemetry + Jaeger
LLM-Metriken	Langfuse (Latenz, Token-Kosten, Halluzinations-Rate)

10. Implementierungs-Roadmap

Meilenstein 1 – Ingest-Pipeline & Vektordatenbank (Wochen 1–6)

Ziel: Dokumente einlesen, chunken, embedden, in pgvector speichern

Aufgaben:

1. PostgreSQL + pgvector Setup (Docker Compose)
2. MinIO-Einrichtung für Dokumentenspeicher
3. Apache Tika + Docling Integration (PDF/DOCX/HTML/RTF)
4. Textnormierungs-Pipeline (Python)
5. Metadaten-Schema implementieren (Pydantic-Modelle)
6. Chunking-Pipeline (spaCy-basiert, strukturell + semantisch)
7. Embedding-Pipeline (FastEmbed, `multilingual-e5-large`)
8. pgvector-Schema anlegen (DDL)
9. Airflow DAG für Ingest-Pipeline
10. Unit-Tests & Integrationstests

Stack: FastAPI, Apache Tika, Docling, spaCy, FastEmbed, PostgreSQL, pgvector, Airflow, MinIO

Meilenstein 2 – NLP & SVO-Extraktion (Wochen 5–10)

Ziel: Subjekte, Objekte, Prädikate und Normtypen aus normativen Texten extrahieren

Aufgaben:

1. spaCy-Pipeline mit `de_core_news_lg` einrichten
2. Dependency-Parser für SVO-Extraktion implementieren
3. Normtyp-Klassifikator (Pflicht/Ermessen/Verbot/Frist) trainieren/fine-tunen auf `deepset/gbert-large`
4. Flair NER für Verwaltungsrecht-Entitäten (Behörden, Rollen, Fristen)
5. SVO-Output in Chunk-Metadaten (JSONB) persistieren
6. Knowledge-Graph-Aufbau (Apache Jena Fuseki)
7. RDF-Triples aus SVO-Extraktion generieren
8. Evaluationsset mit annotierten Normtexten

Stack: spaCy, Flair, Hugging Face Transformers, Apache Jena, SPARQL

Meilenstein 3 – RAG-Engine & Prompt-Suite (Wochen 9–16)

Ziel: Vollständiger RAG-Zyklus für Informationsmodell-Generierung

Aufgaben:

1. LlamaIndex mit pgvector-Integration
2. Hybrid-Retriever (pgvector cosine + PostgreSQL FTS)
3. Re-Ranker-Integration (`ms-marco-MiniLM`)
4. Parent-Child-Retrieval konfigurieren
5. Ollama-Integration (lokales LLM, Mistral 7B)
6. Structured Output mit Pydantic v2 (JSON-Mode)
7. Prompt-Suite PS-01 bis PS-10 implementieren
8. Traceability-Mapper (Claim → Chunk → Dokument)
9. Halluzinations-Detektor (Citation-Grounding Check)
10. Langfuse-Integration für LLM-Observability

Stack: LlamaIndex, Haystack, Ollama/vLLM, Mistral 7B, Pydantic v2, Langfuse

---

Meilenstein 4 – Informationsmodell & Human-in-the-Loop (Wochen 14–20)

Ziel: Vollständiger Phase-1-Prozess mit Fachaufsicht-Workflow

Aufgaben:

1. IM-Schema (YAML/JSON-LD) definieren und implementieren
2. IM-Generator-Agent (LlamaIndex Agent Framework)
3. Airflow DAG für iterativen IM-Prozess
4. Keycloak-Integration (OIDC, Rollen: Analyst, Fachaufsicht, Admin)
5. MN-Pflege-Portal MVP (Next.js):
   - IM-Ansicht mit Quellverweisen
   - Kommentar- und Korrektur-Funktion
   - Freigabe-Workflow (Approve/Reject/Request-Changes)
6. Audit-Log (PostgreSQL: `im_review_log`)
7. IM-Versionierung (MinIO + Git-ähnlicher Versionsbaum)
8. Ergänzungsdokument-Feedback-Loop

Stack: Next.js, FastAPI, Keycloak, Airflow, MinIO, PostgreSQL

Meilenstein 5 – Phase 2: Artefakt-Generierung (Wochen 19–26)

Ziel: Aus bestätigtem IM technische Artefakte automatisiert generieren

Aufgaben:

1. Architektur-Agent implementieren (LlamaIndex Agent + Tools)
2. Factory Prompt Pack (DSL/Schema-Vorgaben)
3. SQL-DDL-Generator (PostgreSQL-Schema aus IM)
4. OpenAPI 3.1 Generator (REST-Schnittstellen)
5. Validierungsregeln-Generator (JSON / DMN 1.3)
6. XÖV-Schema-Generator (XML/XSD)
7. Register-Blueprint Generator (YAML Registerfactory-Format)
8. Artefakt-Validierung (Linting, Konsistenz-Checks)
9. Artefakt-Store (MinIO mit Versionierung)
10. Fachverfahrens-Agent (MVP): Antrag → Validierung → Register

Stack: LlamaIndex, Jinja2 (Template-Engine), xmlschema, pydantic, MinIO

Meilenstein 6 – Produktionshärtung & Föderalisierung (ab Woche 24)

Ziel: Plattform produktionsreif, skalierbar, behördenübergreifend betreibbar

Aufgaben:

1. Kubernetes-Deployment (K3s für On-Premise)
2. Prometheus + Grafana + Loki Setup
3. Multi-Tenancy (Mandantentrennung je Behörde)
4. EUR-Lex / ELI Connector (Normen-Import aus EU-Datenbanken)
5. Sicherheitshärtung (OWASP, BSI Grundschutz)
6. Performance-Optimierung (pgvector IVFFlat-Tuning, LLM-Batching)
7. Dokumentation (Betrieb, API, Entwicklung)
8. Schulungsunterlagen Fachaufsicht

---

11. Qualitätssicherung & Evaluierung

RAG-Qualitätsmetriken

Metrik	Beschreibung	Zielwert
Retrieval Recall@K	Werden relevante Chunks gefunden?	> 0,90
Answer Faithfulness	Ist die Antwort durch Quellen belegt?	> 0,95
Context Precision	Sind alle retriegelten Chunks relevant?	> 0,80
Traceability Coverage	% der IM-Claims mit Normreferenz	100%
Halluzinationsrate	% unbelegt behauptete Fakten	< 0,02

Evaluierungsframework: RAGAS (Open Source, Python) + manuelle Annotation durch Juristen

IM-Qualitätssicherung

- **Vollständigkeitsprüfung:** Alle PS-Prompt-Ergebnisse vorhanden?
- **Konsistenzprüfung:** Keine widersprüchlichen Regeln oder Typen?
- **Normreferenz-Validierung:** Jede Aussage mit gültigem `norm_reference`?
- **Fachaufsicht-Review:** Stichproben-Prüfung durch Juristen/Fachexperten

---

12. Sicherheit & Datenschutz

Anforderung	Maßnahme
Zugriffskontrolle	Keycloak RBAC (Analyst, Fachaufsicht, Deployer, Admin)
Datenverschlüsselung	TLS 1.3 in Transit; AES-256 at Rest (MinIO)
Audit-Trail	Unveränderliche Log-Tabelle mit User-ID + Zeitstempel
Mandantentrennung	Row-Level Security in PostgreSQL je Behörde
BSI Grundschutz	Härtung nach BSI IT-Grundschutz (OPS.1.1.3)
DSGVO	Kein Training auf personenbezogenen Daten; LLMs lokal
Keine Cloud-APIs	Alle LLMs On-Premise (Ollama/vLLM); kein OpenAI

---

13. Wiederverwendbarkeit & Skalierung

Das System ist von Grund auf **fachagnostisch und mandantenfähig** konzipiert:

- **Template-basierte Prompt-Suite:** Neue Register = neues Quelldokument + gleiche PS-01–PS-10 Pipeline
- **Pluggable Parsers:** Neue Dokumentformate über Tika-Plugin-Architektur
- **Schema-Registry:** IM-Schemas und Factory-DSLs in einer zentralen Registry (MinIO)
- **Horizontal skalierbar:** Ingest-Pipeline als Microservice auf K8s; LLM-Serving über vLLM mit GPU-Sharding
- **Föderales Betriebsmodell:** Jede Behörde betreibt eigene Mandanten-Instanz; gemeinsamer Normen-Pool über zentralen Knowledge Graph (Bund)

Anhang A: Entscheidungsmatrix Open-Source-Technologien

Kategorie	Gewählte Technologie	Begründung	Alternative
LLM	Mistral 7B / Llama 3.1	Beste DE-Performance, Apache 2.0	Phi-3, Qwen2
Embedding	multilingual-e5-large	Multilingual, hohe Qualität	GBERT-large
RAG Framework	LlamaIndex	Mächtiger als LangChain für komplexe RAG	Haystack
Vector DB	pgvector	Gesetzt; eliminiert separaten Vektorspeicher	Weaviate, Qdrant
Knowledge Graph	Apache Jena	Etabliert, SPARQL 1.1, RDF/OWL	Oxigraph (leichter)
Parsing	Docling + Tika	Docling: beste PDF-Strukturerkennung	Unstructured.io
Orchestrierung	Airflow	Weit verbreitet, gut integrierbar	Prefect, Dagster
Portal	Next.js + shadcn	Produktionsreif, große Community	Nuxt, SvelteKit
Auth	Keycloak	BSI-empfohlen, OIDC/SAML	Authentik
LLM Observability	Langfuse	Einziger OSS-Stack für LLM-Tracing	Phoenix (Arize)

Dieses Dokument beschreibt die fachliche Zielarchitektur und den Implementierungsweg für das Meta-Normen-Register. Es richtet sich an Product Owner, Architekten und Entwicklungsteams im Public Sector. Alle Technologieentscheidungen berücksichtigen die Anforderungen an Datensouveränität, BSI-Konformität und behördenübergreifende Wiederverwendbarkeit.